GUESS for Social Network Analysis

Ego networks, centrality and brokerage, cohesion, roles, homophily, directed and two-mode ties, with three worked case studies

Contents

1 Why networks? 2

2 Ego networks 2
2.1 Draw one person's world Basic . . . 2
2.2 Ego density and effective size Basic . . . 4

3 Centrality and power 4
3.1 Which kind of central? Basic . . . 4

4 Brokerage and structural holes 7
4.1 Find the brokers between groups Intermediate . . . 7
4.2 Brush the brokers out of a chart Intermediate . . . 7

5 Cohesive subgroups 8
5.1 Cores, cliques and cohesive blocks Intermediate . . . 8
5.2 Core and periphery Intermediate . . . 9
5.3 Members of several groups at once Intermediate . . . 11
5.4 Bridges with few ties Intermediate . . . 11
5.5 Communities and the split they predict Basic . . . 12

6 Roles and positions 12
6.1 Find structurally equivalent members Advanced . . . 12
6.2 Do like link to like? Basic . . . 14
6.3 Reciprocity, triads and status Intermediate . . . 14

7 Signed ties 15
7.1 Friends and enemies Intermediate . . . 15

8 Two-mode networks 17
8.1 People and the events they attend Intermediate . . . 17
8.2 Project onto one kind Intermediate . . . 18

9 Diffusion 20
9.1 How far does it spread, and from where? Intermediate . . . 20
9.2 Simple and complex contagion Advanced . . . 21

10 Case studies **21**
10.1 Case study: the karate club split Intermediate 21
10.2 Case study: marriage, money and the Medici Intermediate 23
10.3 Case study: the formal and the informal organisation Advanced 23

11 Capstone **24**
11.1 Capstone: finding the brokers in an organisation Advanced 24

Why networks?

Social network analysis studies people (or teams, firms, countries) through their relations: who advises, befriends, marries, cites or trades with whom. The questions are old. Who holds power? Who connects groups that otherwise don't talk? Do people cluster with others like them? Do roles exist that different people fill in the same way? What the network adds is that the answers depend on *patterns of ties*, not just on each person's attributes.

This book assumes the Kitchen (*The GUESS Kitchen*) for loading, querying and drawing. Its data, in the cookbooks' `data` folder, are classic small networks whose results you can compare with the literature:

file	what it is
`karate.gdf`	Zachary's karate club: 34 members of a university club that split in two after a dispute between the instructor (`m1`) and the club's officer (`m34`). `club` says which side each joined. Edge `weight` is the number of contexts in which two members met.
`florentine.gdf`	Marriage ties among 15 leading families of Renaissance Florence (Padgett).
`southern-women.gdf`	Davis, Gardner and Gardner's two-mode data: 18 women and the 14 social events (`E1`–`E14`) each attended. `kind` is `woman` or `event`.
`adamic-glance.gdf`	Links among 40 influential US political blogs in 2004, with each blog's `leaning`. Directed.
`sample.gdf`	GUESS's own example: an organisation of 402 employees in departments (`dept`), with communication frequency (`freq`) on the ties.

Network data has two shapes. In *sociocentric* (whole-network) data, every tie in a bounded group is recorded, as in the karate club. In *egocentric* data, each respondent describes their own contacts and the ties among them. Most recipes work on both; ego recipes work on either.

Ego networks

Draw one person's world Basic

You have a whole network. **You want** to see it from one member's point of view: their contacts (alters) and how those contacts are tied to each other.

Ingredients: `karate.gdf` · `egoNetwork` · `subgraph`

```
makeFromGDF("karate.gdf")
ego = egoNetwork(m1)
subgraph(ego, "m1's world")
radialLayout(m1)
rescaleLayout(700, 700)
m1.color = red
labelBy(label)
alters = ego.difference([m1])
len(alters), len(alters <-> alters)
```

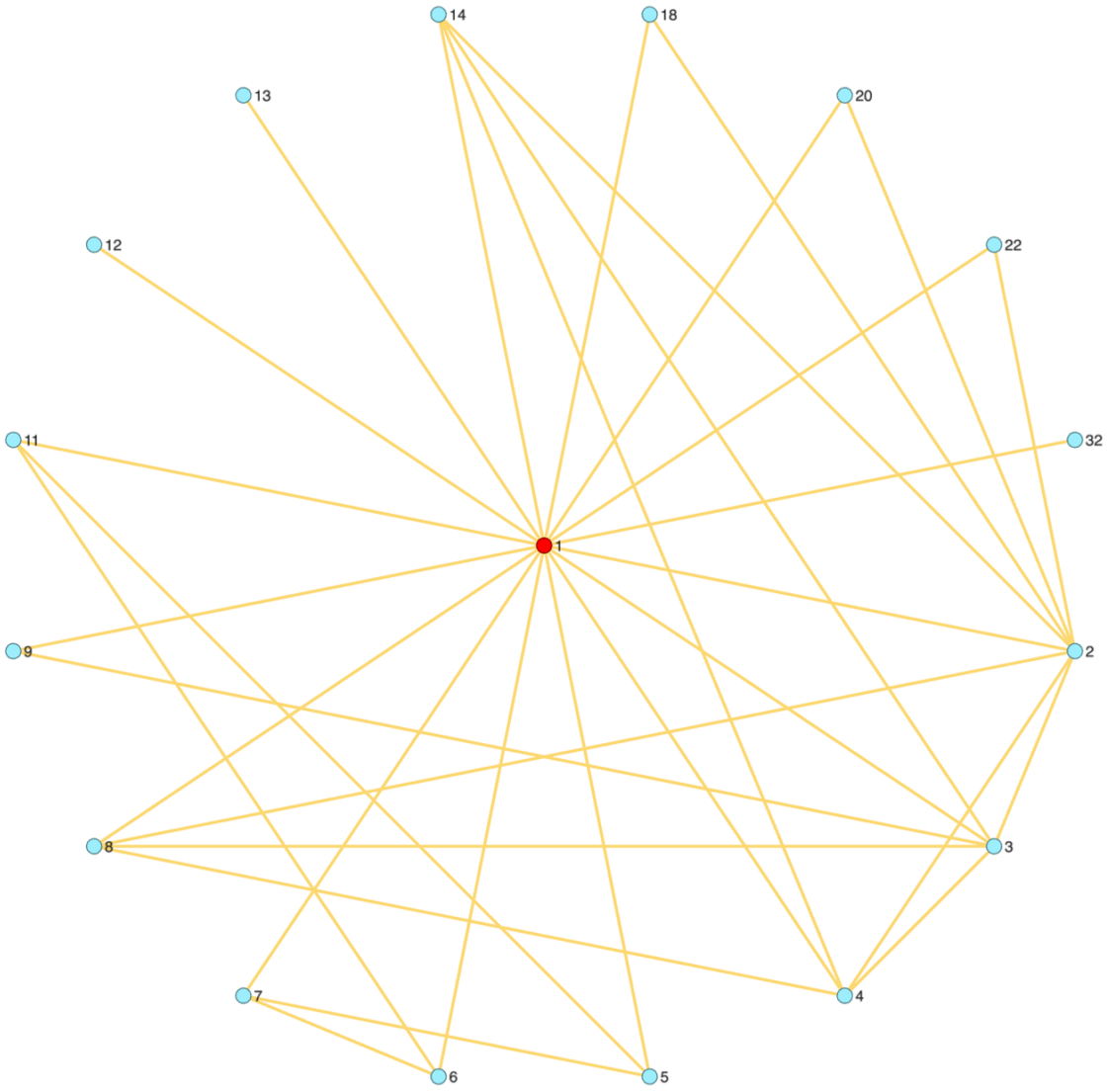


Figure 1: The instructor's ego network

How it works. `egoNetwork(node, depth)` is the node plus everyone within `depth` steps (1 by default). `subgraph` opens it as a graph of its own (`use("main")` goes back), so the layout arranges only the ego network. `showOnly(ego)` would instead hide everything else in place. `alters <-> alters` is the set of ties among the contacts. The instructor has 16 contacts with 18 ties among them out of 120 possible, so his contacts mostly don't know each other: he is the hub that holds his side of the club together.

Variations. `egoNetwork(m1, 2)` adds friends of friends. For egocentric survey data, load each respondent's network as its own graph (`newGraph`, then `use` to switch between them) or keep them in one graph with an `ego` field.

Ego density and effective size Basic

You want numbers that summarise an ego network, to compare people.

```
makeFromGDF("karate.gdf")
alters = egoNetwork(m34).difference([m34])
ties = len(alters <-> alters)
possible = len(alters) * (len(alters) - 1) / 2
ties / possible                          # ego density
m34.effsize, m34.constraint
```

How it works. *Ego density* is the share of possible ties among the alters that exist: high when the alters form a closed group, low when the ego bridges otherwise separate people. Burt's *effective size* (`effsize`) counts alters minus their redundancy: 10 contacts who all know each other are effectively fewer than 10. *Constraint* (`constraint`) is high when an ego's contacts are few and tied to each other, which leaves the ego no alternatives. Both are built-in node fields, computed for every node on first use, so `sortBy(constraint)[:5]` gives the least constrained people.

Centrality and power

Which kind of central? Basic

You want to know who is important, and to understand why the measures disagree.

```
makeFromGDF("karate.gdf")
for n in sortBy(betweenness)[-5:]:
    print(n.label, n.degree, round(n.betweenness, 1), round(n.eigenvector, 3))
```

How it works. Each measure models a different idea of importance. *Degree* is activity: how many ties. *Betweenness* is control: how often a node lies on the shortest paths between others, so its removal would lengthen them. *Closeness* is reach: how few steps to everyone. *Eigenvector* centrality is status: ties to people who are themselves well tied. The instructor and the officer top every list here. Further down the lists differ. Member `m32` has only 6 ties but almost the betweenness of `m33`, who has 12, because `m32`'s friends are on both sides of the club.

Variations. Eigenvector centrality gives zero to anyone no one points to in a directed network, and to everyone they point to. *Katz centrality* gives every node a free amount β as well, so it works on any network:

```
f, limit = katzCentrality(alpha=0.1)
limit                                    # alpha must stay below this: 1/κ₁
[n.label for n in sortBy(katz)[-3:]]
```

With α near 0 it ranks almost by degree; as α approaches the limit it ranks almost like eigenvector centrality.

Pitfalls. Betweenness and closeness assume things flow along shortest paths. Gossip, which wanders, is better modelled by `rwbetweenness` (random-walk betweenness) or `pagerank`. ## The Medici: power from position Intermediate

You have the Florentine marriage network. **You want** to see why one family dominated without having the most of anything.

Ingredients: `florentine.gdf` · `betweenness` · `scatterLayout`

```
makeFromGDF("florentine.gdf")
kkLayout()
sizeBy(betweenness, 10, 50)
colorBy(degree, lightgray, red)
labelBy(name)
Medici.degree, Strozzi.degree
round(Medici.betweenness, 1), round(Strozzi.betweenness, 1)
```

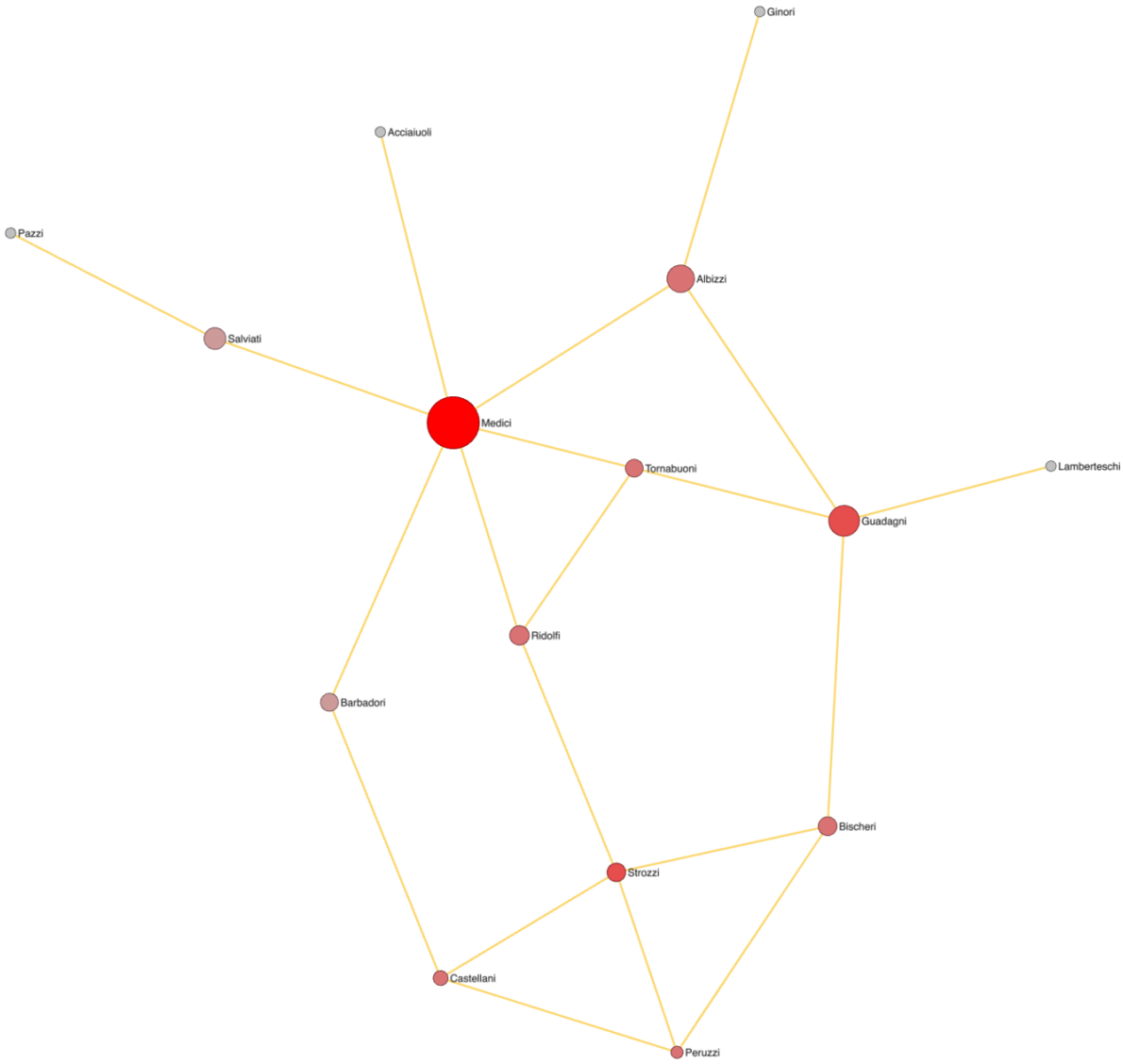


Figure 2: Florentine marriages, sized by betweenness

How it works. The Medici have six marriage ties, the Strozzi four, but the Medici lie on nearly half of all shortest paths between families. They married into families that weren't tied to each other, so other families reached each other through them. This is Padgett's classic argument: the Medici's power came from bridging, not from size.

Variations. `scatterLayout(degree, betweenness)` with labels on puts each family at its degree (x) and betweenness (y). Families far above the diagonal are brokers.

Brokerage and structural holes

Find the brokers between groups Intermediate

You have an organisation with formal departments. **You want** the people who connect departments.

Ingredients: `sample.gdf` · `interEdges` · `constraint` · `cutNodes`

```
makeFromGDF("sample.gdf")
crossing = interEdges(dept)
crossing.color = red
addNodeField("ndepts", INTEGER, 0)
for n in g.nodes: n.ndepts = len(set(m.dept for m in n.getNeighbors()))
brokers = (ndepts >= 3)
brokers.color = orange
sortBy(brokers, constraint)[:5]
len(crossing), len(brokers)
```

How it works. A *structural hole* is a gap between two groups with no direct ties. People who span it see information from both sides early and can broker between them. This recipe stores, for each person, how many departments their contacts belong to (`ndepts`). Among those spanning three or more, the least constrained are the strongest brokers. `cutNodes()` gives the extreme case: people whose departure would disconnect part of the organisation.

Pitfalls. A tie across a hole is valuable only if the groups differ in what they know. Check that the groups are meaningful before celebrating the brokers.

Brush the brokers out of a chart Intermediate

You have the brokers from the last recipe. **You want** to see how they stand out, and pick them by eye.

Ingredients: `sample.gdf` · `nodeData` · an Altair brush

```
makeFromGDF("sample.gdf")
addNodeField("ndepts", INTEGER, 0)
for n in g.nodes: n.ndepts = len(set(m.dept for m in n.getNeighbors()))
corner = alt.selection_interval()
alt.Chart(nodeData("constraint", "betweenness", "ndepts")).mark_point().encode(
    x="constraint", y="betweenness", color="ndepts:O",
    tooltip="name").add_params(corner)
```

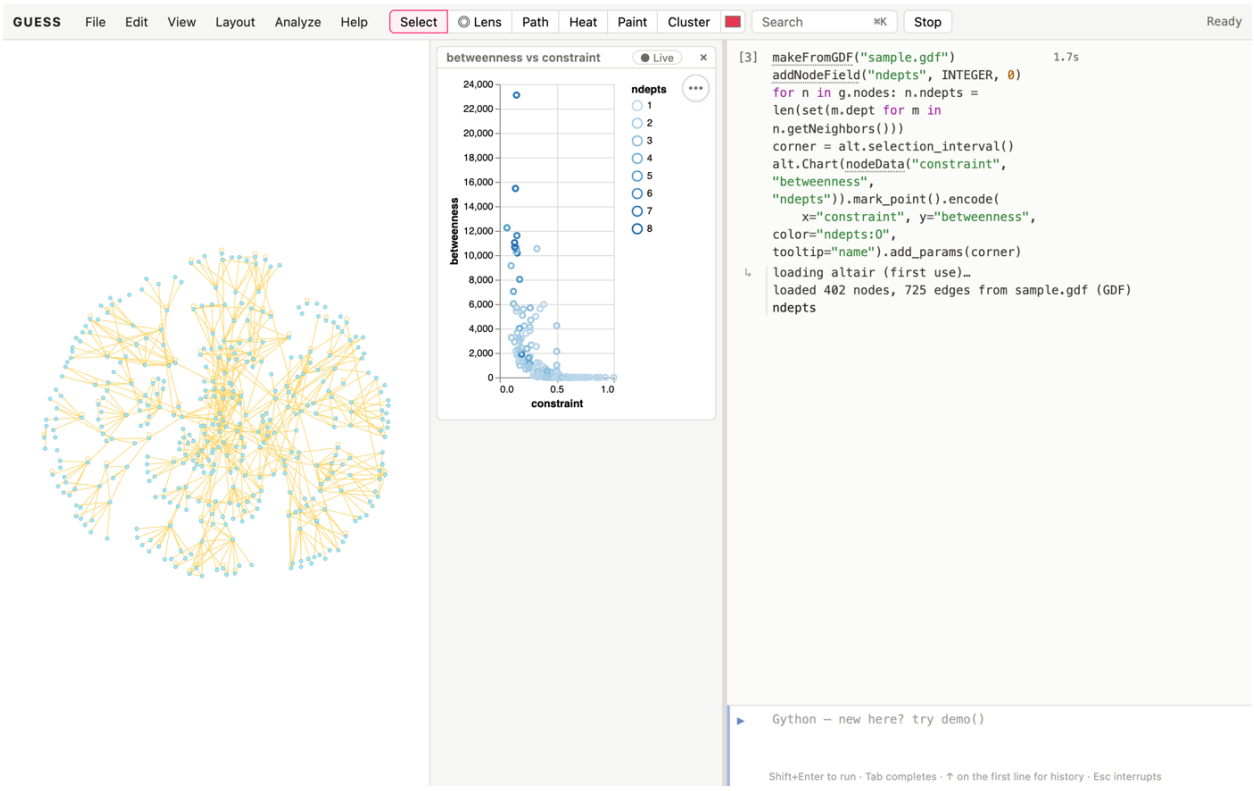


Figure 3: Constraint against betweenness, coloured by how many departments each person's contacts span

How it works. Brokers sit in the top-left corner: their contacts don't know each other (low constraint), and many shortest paths run through them (high betweenness). v30 has the highest betweenness, and v73 the lowest constraint (0.064, with 19 contacts). The colours show `ndepts`: the 25 people whose contacts span three or more departments average a betweenness of 6720, against 482 for everyone else. Drag a box over the corner. The people inside light up in the graph while you drag, and are selected when you let go, so `sel.color = orange` marks them. Pointing at a single dot shows who it is.

Pitfalls. 23 people have no constraint (no contacts, or only one), so they're missing from the chart. A corner is a judgement call: the brush shows exactly which people your judgement includes.

Cohesive subgroups

Cores, cliques and cohesive blocks Intermediate

You want the tight-knit groups inside a network.

```
makeFromGDF("karate.gdf")
frLayout()
colorBy(coreness, lightgray, red)
[sorted(n.label for n in c) for c in largestCliques()]
cohesiveBlocks()
```

How it works. A *clique* is a group where everyone is tied to everyone: the strictest definition,

and brittle (one missing tie breaks it). A *k-core* relaxes it: everyone has at least k ties inside the group. `coreness` gives each member's deepest core, so colouring by it shows the club's dense centre. *Cohesive blocks* (Moody and White) are nested groups that stay connected after removing any $k-1$ members, so they catch groups held together by many independent paths.

Variations. `kComponents(k)` finds the *k-components*: groups in which every pair is joined by k independent paths (no shared members), largest first, in the field `kcomponent`. A k-core only counts ties, so two dense groups joined through a single member can form one k-core; they can't form one k-component:

```
[len(c) for c in kComponents(2)]          # bicomponents: 28 and 6 members
[len(c) for c in kComponents(3)]
```

Core and periphery Intermediate

You want to know whether a network is organised around a dense core that everyone else hangs off, rather than around separate groups.

```
makeFromGDF("karate.gdf")
frLayout()
corePeriphery()                           # {'core': 5, 'fit': 0.43}
colorize(core)
sizeBy(degree, 10, 40)
showMatrix(order=core)
```

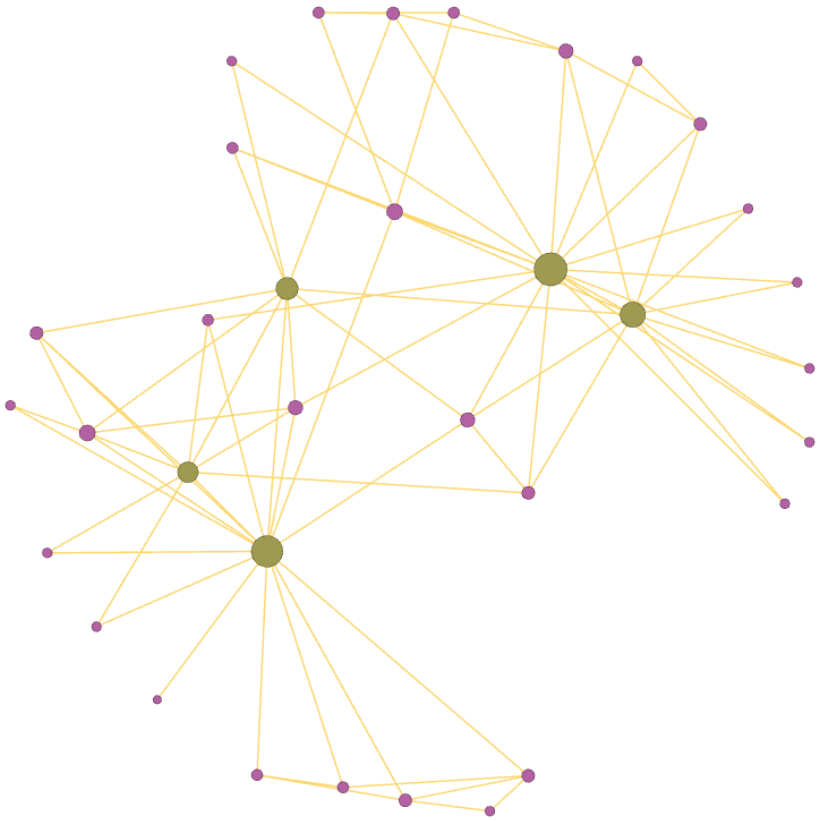


Figure 4: Five members form the karate club's core

How it works. `corePeriphery()` looks for the split that best matches an ideal pattern: every pair that includes a core member is tied, and no pair of peripheral members is (Borgatti and Everett). It stores 1 in `core` for core members and returns the core's size and the correlation with the ideal. In the matrix, ordered by `core`, the core's rows and columns are dense and the periphery's corner is nearly empty. In the karate club the core is the instructor, the officer and three of their closest associates: `m1`, `m2`, `m3`, `m33`, `m34`.

Variations. A two-group block model without degree correction finds the same split here, because it groups nodes by how many ties they have:

```
fitBlockModel(2, degreeCorrected=false)
compareClusters(core, sbm_class)          # 1.0: the same five members
```

Pitfalls. Every network has a best core, so look at the fit and the matrix before calling it core–periphery. A network with two communities can also have a "core" of their leaders, as the karate club does.

Members of several groups at once Intermediate

You want groups that may overlap, because people belong to more than one.

Ingredients: `karate.gdf` · `cliquePercolationClusters` · `showOverlaps`

```
makeFromGDF("karate.gdf")
frLayout()
cs = cliquePercolationClusters(3)
showOverlaps(cs, hulls=true)
[(n.label, n.cpm_classes) for n in (cpm_count > 1)]
```

How it works. Clique percolation (Palla et al., 2005) chains triangles that share an edge into one community. Community detection usually gives each person one group; here a person can sit in several. The club splits into a large core of 25 and two small groups, one on each side of the split. Two people are in two communities at once: the instructor (`m1`), who also anchors a small circle of his own students, and `m32`, a member of the officer's side who also belongs to a three-person cluster. `showOverlaps` colours each community, draws the members of several as pies, one slice per group, and with `hulls=true` shades each group.

Variations. `cliquePercolationClusters(4)` asks for groups built from 4-cliques: smaller, tighter groups. `linkCommunities()` clusters ties rather than people (Ahn et al., 2010), which suits people who belong to many contexts, such as family, work and sport. Each tie gets one context (`link_class`) and each person all of theirs (`link_classes`).

Pitfalls. People in no community (`cpm_count == 0`) are not isolates: they have ties, just none that close a triangle.

Bridges with few ties Intermediate

You want the people who connect parts of a network without being busy hubs themselves.

Ingredients: `karate.gdf` · `bridging` · `betweenness`

```
makeFromGDF("karate.gdf")
[(n.label, round(n.bridging, 1), n.degree) for n in highest(bridging, 3)]
```

```
[(n.label, round(n.betweenness, 1), n.degree) for n in highest(betweenness, 3)]
colorBy(bridging, lightgray, red)
```

How it works. `betweenness` rewards lying on shortest paths, and the busiest nodes win: the instructor (`m1`) and the officer (`m34`). Bridging centrality (Hwang et al., 2006) multiplies betweenness by a *bridging coefficient* that is high when a person's contacts are better connected than they are. The top bridge is `m20`, with only three ties: one each to the instructor, his student `m2` and the officer. Next is `m9`, tied to both leaders and to the officer's side. Those are the people through whom the two factions could still talk.

Variations. `weightedBridging()` treats heavy ties as short ones, for networks whose weights mean closeness. `bridging_coef` on its own is the coefficient without betweenness.

Communities and the split they predict Basic

```
makeFromGDF("karate.gdf")
frLayout()
louvainClusters()
colorize(louvain_class)
shapeBy(club)
compareClusters(louvain_class, club)
```

How it works. Community detection finds groups denser inside than between. Here, colour is the found community and shape is the side each member actually joined after the split. The two largely coincide: the fault line was visible in the friendships before the club broke up. See the case study for a sharper version.

Roles and positions

Find structurally equivalent members Advanced

You want *roles*: sets of people tied in the same way to the same others, whether or not they're tied to each other. Leaders who share the same followers, or followers who share the same leaders, are examples.

Ingredients: `karate.gdf` · `blockmodel` · `collapseBy`

```
makeFromGDF("karate.gdf")
frLayout()
blockmodel(4)
colorize(position)
legendFor(position)
```

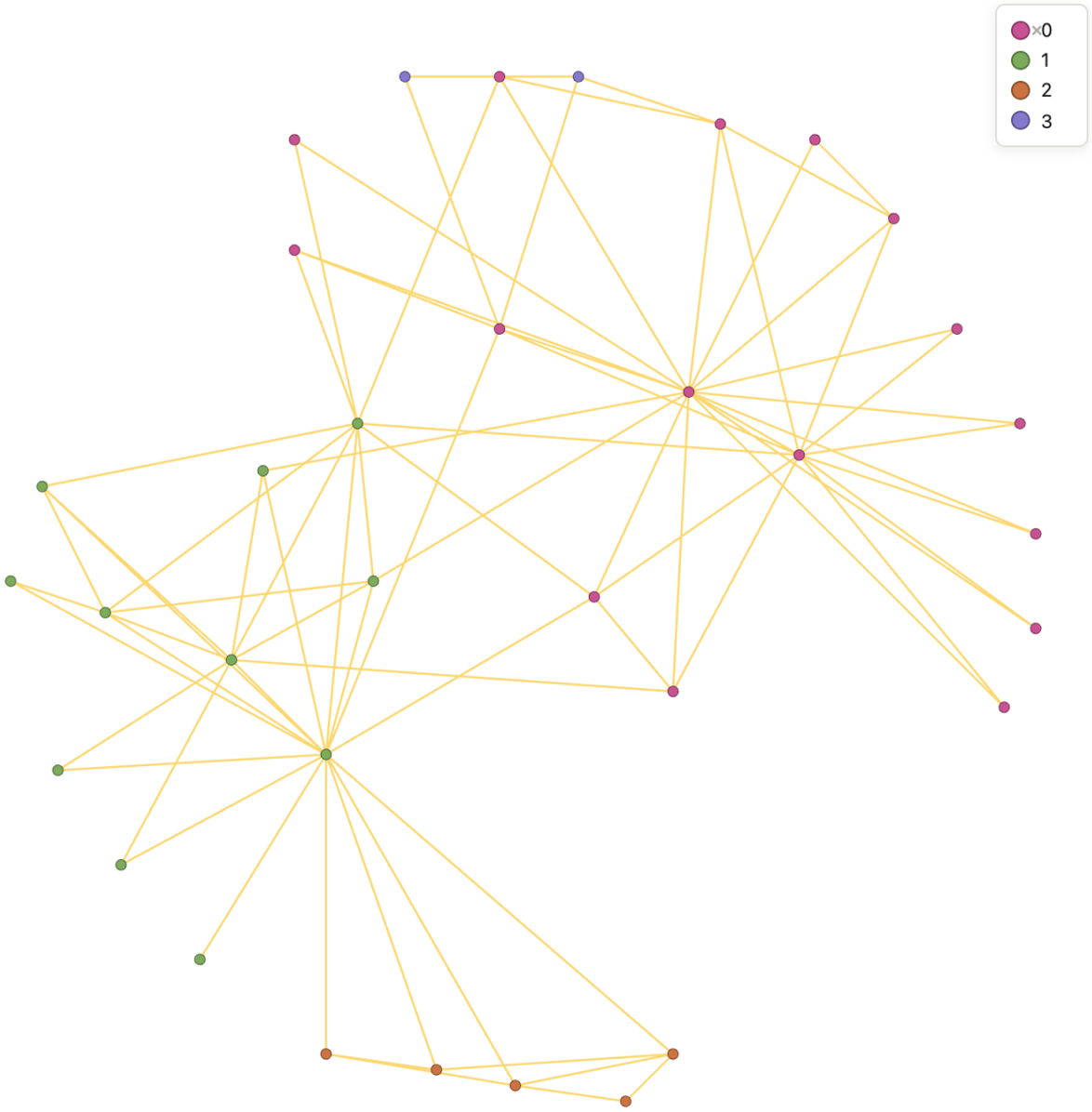


Figure 5: Four positions in the karate club

How it works. `blockmodel(k)` compares every pair of members' tie profiles (whom they're tied to) by correlation and groups the most similar into k positions, stored in `position`. The printed *image matrix* gives the density of ties from each position to each other. A dense diagonal cell means a cohesive position. A dense off-diagonal cell with a sparse diagonal means a role that serves another, such as followers who all tie to the same leaders but not to each other. In the karate club, two large positions follow the two sides. A third position, members 5, 6, 7, 11 and 17, is a satellite of the instructor: tied to him and to each other but to almost no one else. A fourth is the pair 25 and 26, who share the same few officer-side contacts.

Variations. `collapseBy(position)` draws the reduced graph: one node per position. Try several *k*: there is no single right number, only more or less useful simplifications.

Structural equivalence asks for the *same* contacts. Two team leaders in different departments share no contacts at all, yet play the same role. *Regular equivalence* calls two members similar when their contacts are similar, counting paths of every length (Leicht, Holme and Newman):

```
top = similarTo(m34, method="regular", alpha=0.9, top=3)
[n.label for n in top]
colorBy(similarity, white, red)
```

`alpha` (below 1) weighs longer paths; the score is in `similarity`.

Pitfalls. Communities and positions answer different questions. A position can hold people with no ties to each other at all (the followers of one leader), and a community can mix leaders and followers. # Homophily and mixing

Do like link to like? Basic

You have a network and an attribute. **You want** to know whether ties stay within groups.

Ingredients: `adamic-glance.gdf` · `assortativity` · `mixingMatrix`

```
makeFromGDF("adamic-glance.gdf")
colorize(leaning)
interEdges(leaning).color = gray
assortativity(leaning)
mixingMatrix(leaning)
```

How it works. `assortativity(field)` is near 1 when ties stay within values, 0 when they mix at random, and negative when they cross values. `mixingMatrix` shows the counts behind it. For these 40 blogs the assortativity is about 0.29: positive, but 210 of the 706 links cross sides. These are the most prominent blogs, which watched and answered each other. Adamic and Glance's study of many more blogs found the two sides far more separated.

Variations. A numeric field is treated as a quantity: `assortativity(age)` is the correlation of the values at the two ends of each tie. If the numbers are codes for categories (a grade, a class, a department number), say so, and the categorical measure is used instead:

```
makeFromGDF("karate.gdf")
addNodeField("side_code", INTEGER, 0)
for n in g.nodes:
    n.side_code = 1 if n.club == "Mr Hi" else 2
assortativity(side_code, categorical=true)
```

Pitfalls. Homophily has two explanations that the network alone can't separate: people choose similar others (*selection*), or tied people grow alike (*influence*). Telling them apart needs data over time. # Directed ties

Reciprocity, triads and status Intermediate

You have directed ties (advice, citation, links). **You want** to know how often they're returned and whether they form a hierarchy.

```
makeFromGDF("adamic-glance.gdf")
reciprocity()
mutualEdges().color = red
triadCensus()
sortBy(authority)[-3:], sortBy(hub)[-3:]
```

How it works. *Reciprocity* is the share of directed ties returned. The *triad census* counts the 16 kinds of three-node configuration. Many transitive triads (A→B, B→C, A→C) and few cycles (A→B→C→A) suggest a ranking. Many cycles suggest exchange among equals. Among the blogs, 61% of links are returned, and transitive triads outnumber cycles 455 to 29: a pecking order, not a ring. `authority` and `hub` (HITS) split status into being pointed to by good hubs (authorities) and pointing to good authorities (hubs).

Variations. `rankNodes()` places every node on a ladder so that ties mostly point one way (SpringRank, De Bacco, Larremore and Moore). By default an edge a → b pulls a one step above b, which suits dominance (a beat b) and placement (b hired a's graduates). A link to a blog endorses the blog linked, so here the edges are read the other way with `endorsements=true`. It stores the position in `rank` and returns the share of ties that agree with the ladder:

```
rankNodes(endorsements=true)              # 0.65 of the links agree with the ladder
[n.name for n in sortBy(rank)[-3:]]         # the top: Wonkette, JuanCole,
 ↪  BlogJohnKerry
```

Signed ties

Friends and enemies Intermediate

You have ties that are positive or negative (friend or rival, trust or distrust). **You want** to know whether the group is stable, and if not, which ties are the trouble.

```
newGraph("rivals")
for v in ["Ann", "Bob", "Cy", "Dee", "Eve", "Fay", "Gus"]:
    addNode(v)
Edge.sign = 1
ties = [("Ann", "Bob", 1), ("Ann", "Cy", 1), ("Bob", "Cy", 1), ("Dee", "Eve", 1),
 ↪  ("Eve", "Fay", 1),
        ("Dee", "Fay", 1), ("Fay", "Gus", 1), ("Cy", "Dee", -1), ("Bob", "Eve",
 ↪  -1), ("Ann", "Fay", -1),
        ("Cy", "Gus", -1), ("Gus", "Dee", -1)]
for a, b, sg in ties:
    e = addEdge(g.getNodeByName(a), g.getNodeByName(b))
    e.sign = sg
circleLayout()
g.nodes.width = g.nodes.height = 30
labelBy(name)
r = structuralBalance(sign)
colorize(camp)
(sign < 0).linestyle = "dashed"
```

```
r["frustrated"].color = red
r["balanced"], r["unbalanced"]
```

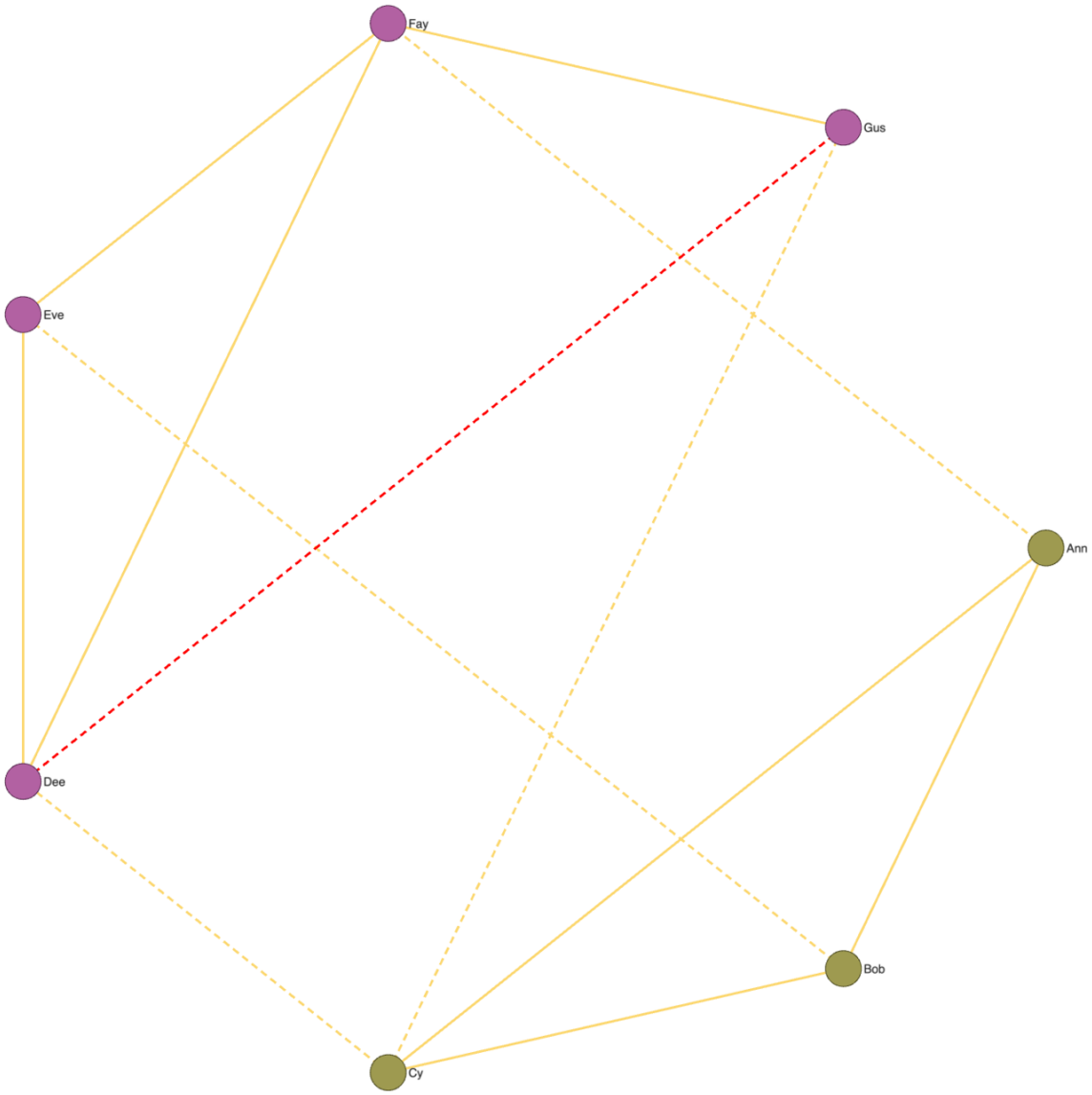


Figure 6: Two camps, with the one tie that doesn't fit in red

How it works. A network of friends and enemies is *balanced* (Heider, Cartwright and Harary) when every cycle has an even number of negative ties: "the enemy of my enemy is my friend". Harary showed that a balanced network splits into at most two camps, with friendship inside each and enmity between. `structuralBalance(sign)` reads an edge field of $+1$ and -1 (or "+" and "−"), puts the best two-camp split in `camp`, and returns whether the network is balanced, the triangles with an odd number of negative ties, and the ties that disagree with the camps. Here Gus is a friend of Fay but an enemy of Dee, who is Fay's friend. Two triangles are unbalanced (Dee, Fay, Gus, with

one negative tie, and Cy, Dee, Gus, with three), and both run through the same tie: Gus–Dee, the one the camps can't place.

Pitfalls. Balance theory predicts which configurations are under strain, not that they will change. Real signed networks are rarely perfectly balanced; compare the number of unbalanced triangles with what the same signs shuffled at random would give.

Two-mode networks

People and the events they attend Intermediate

You have a two-mode (bipartite) network: people and the groups, events or organisations they belong to. **You want** to see it, and then to study the people through what they share.

Ingredients: `southern-women.gdf` · `bipartiteLayout` · `project`

```
makeFromGDF("southern-women.gdf")
bipartiteLayout(kind)
colorize(kind)
labelBy(name)
```

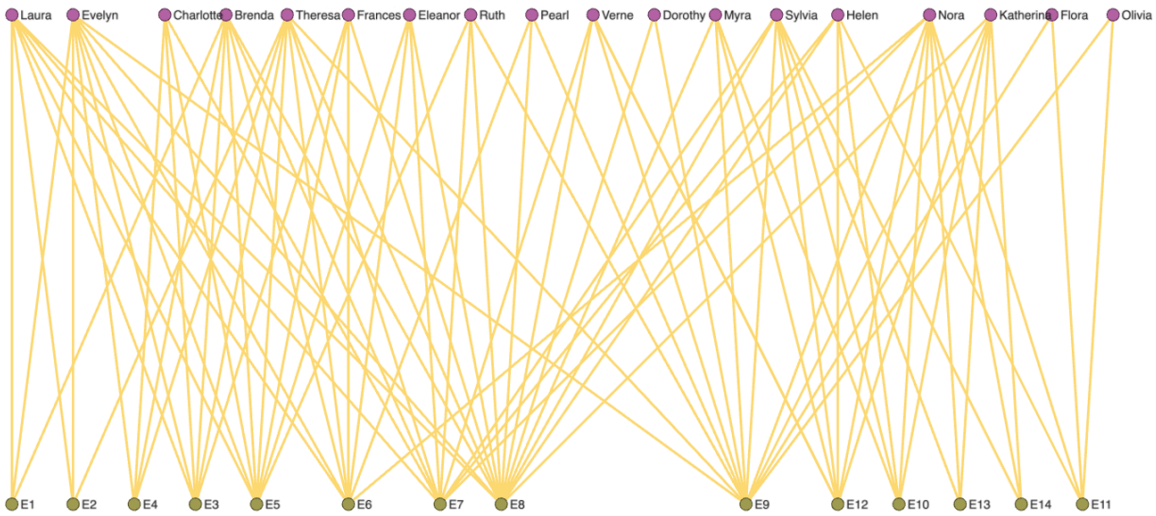


Figure 7: Women and the events they attended

How it works. `bipartiteLayout(field)` draws the two kinds of nodes in two rows and orders each row to reduce crossings, so groups of women who attend the same events cluster together.

Project onto one kind Intermediate

```
makeFromGDF("southern-women.gdf")
project(kind, "woman", method="newman")
frLayout()
```

```
sizeBy(affiliations, 8, 30)
sizeBy(weight, 1, 8)
louvainClusters(); colorize(louvain_class)
labelBy(name)
```

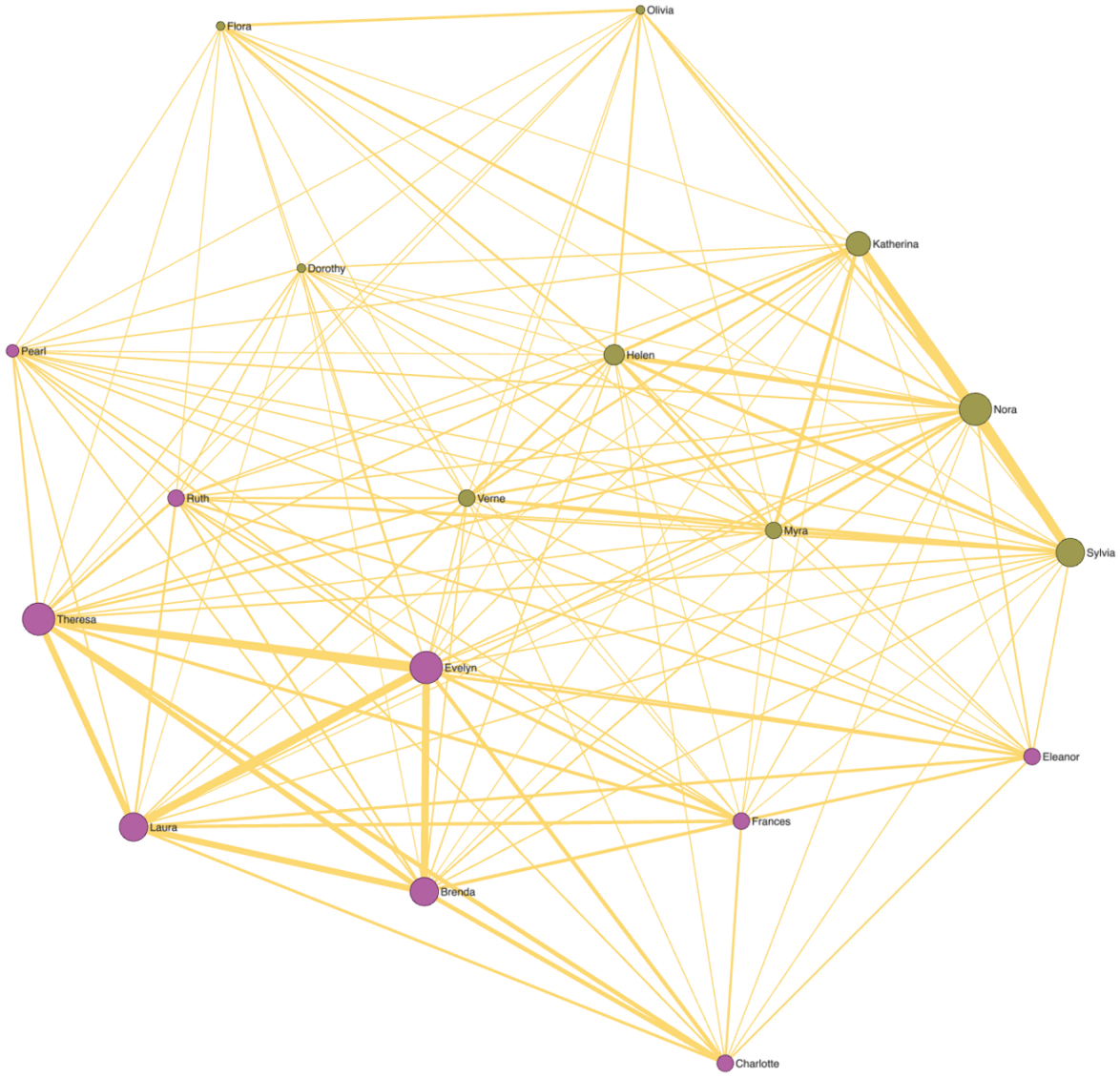


Figure 8: The women's network, projected

How it works. `project(kind, "woman")` makes a new graph of just the women, tying two when they attended an event together, and switches to it (the two-mode graph stays: `use("main")` goes back). Every edge stores `shared`, the number of events in common, and `weight` according to `method`:

method	weight	use when
`"count"`	the events shared	events are similar in size
`"jaccard"`	shared ÷ events either attended	some people attend everything
`"newman"`	each shared event adds 1/(its size − 1)	big events say less about a tie than small ones

`affiliations` is each woman's number of events. The communities found in the projection recover the two cliques that Davis and the Gardners described.

Variations. `project(kind, "event")` gives the dual: events tied by the women they share, which shows how the social calendar hung together.

Pitfalls. Projection makes dense, clique-ridden networks (everyone at one event becomes a clique), so degree and clustering in the projection are inflated. Compare with the two-mode graph, and prefer weighted measures.

Diffusion

How far does it spread, and from where? Intermediate

You want to know how far something (a rumour, an infection, an innovation) travels through a network, and how much it matters where it starts.

Ingredients: `karate.gdf` · `spread`

```
makeFromGDF("karate.gdf")
frLayout()
spread([m1], "SIR", beta=0.15, gamma=0.3, seed=1)
colorBy(infected_at, yellow, red)
(infected_at < 0).color = lightgray
reach = []
for k in range(20):
    spread([m12], "SIR", beta=0.15, gamma=0.3, seed=k, show=False)
    reach.append(len(infected_at >= 0))
sum(reach) / len(reach)
```

How it works. `spread(seeds, model)` simulates step by step. In the SIR model each infected member infects each susceptible friend with probability `beta` per step, and recovers with probability `gamma`. It stores when each member was reached (`infected_at`, −1 if never) and plots the susceptible, infected and recovered counts. Because it's random, one run proves little: the loop repeats it 20 times. Started from the instructor, an outbreak with these parameters reaches about 18 members on average. Started from member 12, who has a single friend, it reaches about 8. Position decides reach.

Variations. `showTime(t, infected_at)` replays a run step by step. Compare `"SI"` (no recovery), and `"cascade"`, where each newly reached member gets one chance to pass it on.

Simple and complex contagion Advanced

```
makeFromGDF("karate.gdf")
spread([m1, m2], "threshold", threshold=0.4)
spread([m1, m34], "threshold", threshold=0.4)
```

How it works. Some things spread through single contacts (a cold). Others need reinforcement from several (an expensive new habit). In the *threshold* model a member adopts only once at least `threshold` of their friends have. Seeded with two close allies, the instructor and member 2, it reaches 14 members, and stalls at the edge of their side of the club. Seeded with the two rival leaders, it reaches 29. Complex contagions need seeds spread across groups, because bridging ties carry them poorly. That's the opposite of the advice for simple contagions, where a single central seed does well.

Case studies

Case study: the karate club split Intermediate

Zachary watched a university karate club for two years, during which a dispute between the instructor and the club's officer split it in two. He asked whether the split could be predicted from the friendships before it.

```
makeFromGDF("karate.gdf")
frLayout()
(m1, m34).color = red
(m1, m34).size = 30
labelBy(label)
```

First, look: the instructor (`m1`) and the officer (`m34`) sit at the centres of two loosely joined halves.

```
fl = minCut(m1, m34)
fl
```

Zachary's own prediction was a minimum cut: the fewest friendship ties that, removed, separate the two leaders. `minCut(a, b)` returns those ties.

```
addNodeField("side", VARCHAR, None)
for n in g.nodes:
    closer = len(shortestPath(n, m1)) <= len(shortestPath(n, m34))
    n.side = "Mr Hi" if closer else "Officer"
compareClusters(side, club, method="ari")
louvainClusters(resolution=0.6)
compareClusters(louvain_class, club, method="ari")
colorize(club)
shapeBy(louvain_class)
```

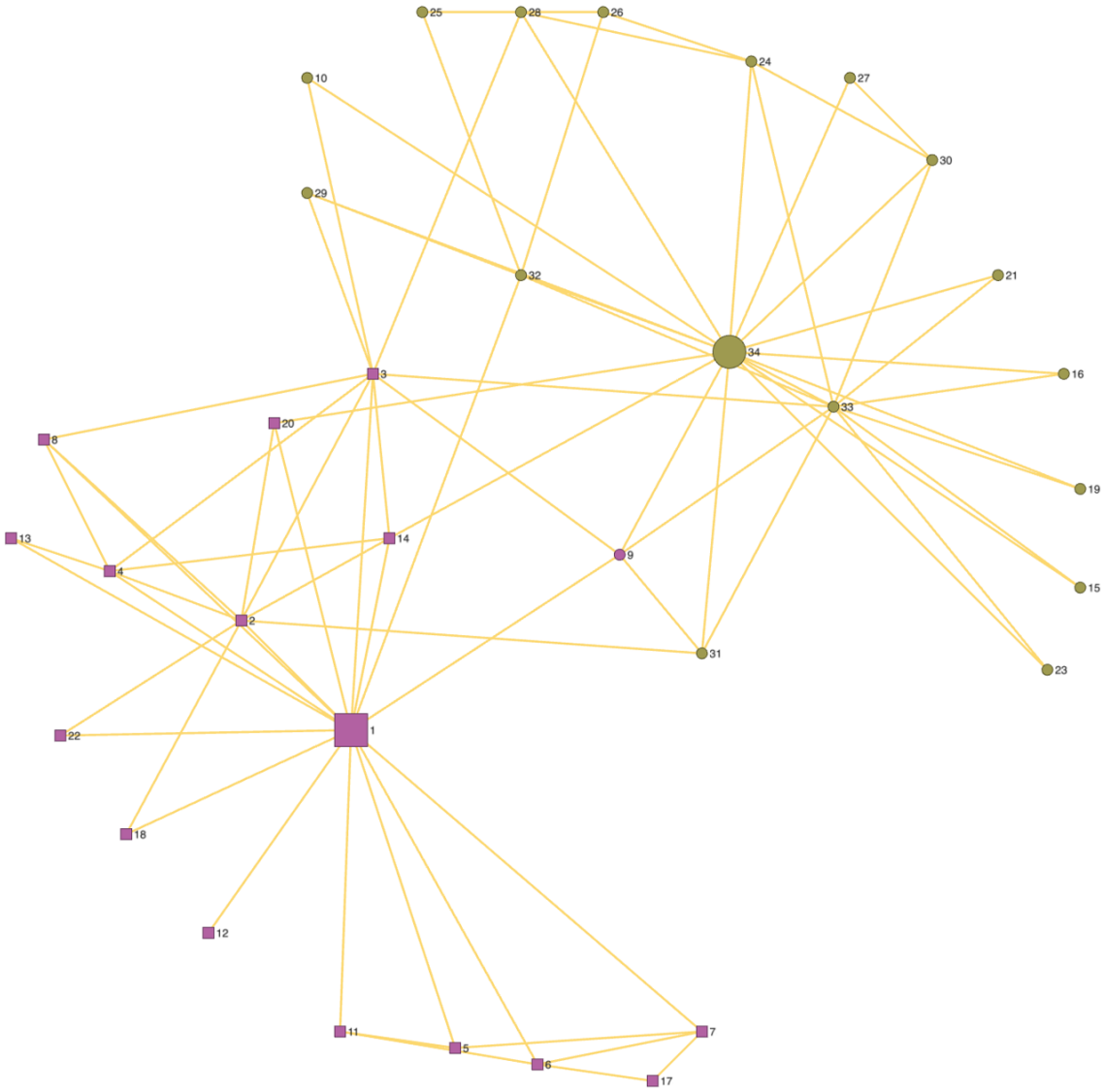


Figure 9: Sides joined (colour) against detected communities (shape)

Reading it. A naive rule (each member follows whichever leader is fewer steps away) gets an adjusted Rand index of 0.67 against the real split. It misplaces members 25, 26 and 32, who are close to the instructor in steps but bound to the officer's side by their friends. Louvain at a coarse resolution (two communities) does better, at 0.88, and gets exactly one member wrong: `m9`. Three of his five friends were on the officer's side, yet he followed the instructor. Zachary's explanation is one no network holds: he was close to his black-belt examination, and staying with the instructor protected years of training. Report the mistakes, not just the score.

A model's view. Instead of a community method, fit a stochastic block model: the two groups that

make the observed friendships most probable.

```
fitBlockModel(2, degreeCorrected=false)   # the plain model
sorted(n.name for n in g.nodes if n.sbm_class == 1)
fitBlockModel(2)                          # degree-corrected (the default)
compareClusters(sbm_class, club)
```

The plain model splits the club into its five best-connected members and everyone else: in that model, members of a group have similar numbers of ties, so hubs group with hubs. Allowing for degree (Karrer and Newman) recovers the two factions, except `m9` and `m10`. That is not a failure of the search: this split has a higher likelihood (−739.4) than the real one (−743.2). The model, too, can't see why `m9` stayed with the instructor.

Case study: marriage, money and the Medici Intermediate

```
makeFromGDF("florentine.gdf")
kkLayout()
labelBy(name)
heatMap(Medici)
for n in sortBy(betweenness)[-4:]:
    print(n.name, n.degree, round(n.betweenness, 1), round(n.constraint, 2))
```

The heat map colours families by marriage distance from the Medici. Eleven of the other fourteen families are within two steps, and several reach each other only through them. Their constraint is the lowest of any family, because their in-laws weren't in-laws of each other. Now remove them and see what's left:

```
ss("with")
remove(Medici)
colorize(component)
graphStats()
```

Computed fields such as `component` and `betweenness` follow the edit: they're recalculated the next time they're read. Without the Medici the network falls into three pieces: the Acciaiuoli alone, the Pazzi and Salviati together, and the other eleven. Their marriages were the glue of the elite, which is why marrying into them was worth so much. `ls("with")` brings them back.

Case study: the formal and the informal organisation Advanced

Every organisation has two structures: the chart (departments, reporting lines) and the network of who actually talks to whom. Where they differ is where coordination happens, or fails.

```
makeFromGDF("sample.gdf")
colorize(dept)
louvainClusters()
compareClusters(louvain_class, dept)
mixingMatrix(dept, show=False)["dept4"]
```

The found communities agree with departments fairly well (normalized mutual information about 0.75, where 1 is identical), but Louvain finds 44 communities for 21 departments: departments

contain several working groups. `mixingMatrix(dept)[d]` shows where one department's ties go.

```
assortativity(dept)
subgraph(freq > 100, "heavy ties")
assortativity(dept)
```

`subgraph(freq > 100, "heavy ties")` opens a graph of just the 367 heavy communication ties and the people they join (`use("main")` goes back). If assortativity by department rose there, departments would be where the real work happens. If it fell, the important work would cross department lines. Here it barely moves (0.77 to 0.76): heavy and light ties follow the chart alike. The exceptions are the brokers (*Find the brokers between groups*): the few people whose heavy ties cross departments.

Capstone

Capstone: finding the brokers in an organisation Advanced

The problem. An organisational consultant wants to find the people who connect departments, and to understand each one's position. Who are they? Which departments do they bridge? How redundant are their contacts? This recipe builds a broker finder on the sample organisation:

- people are coloured by department and sized by how many departments their contacts span;
- a slider sets the minimum span to count as a broker, and the others fade;
- a drop-down focuses on one department and its outside ties;
- a click on a person shows their brokerage profile and highlights their ego network;
- a chart shows how constraint is distributed.

```
makeFromGDF("sample.gdf")
forceAtlas2Layout()
colorize(dept)
addNodeField("span", INTEGER, 0)
for n in g.nodes: n.span = len(set(m.dept for m in n.getNeighbors()) - {n.dept})
sizeBy(span, 4, 26)
plotHistogram(constraint)

def brokers(v):
    g.nodes.labelvisible = false
    picked = (span >= int(v))
    g.nodes.color = "215,215,215"
    colorize(dept)
    (span < int(v)).color = "225,225,225"
    picked.labelvisible = true
    summary.text = f"{len(picked)} people reach {int(v)} or more other
 ↪  departments"

def focus(d):
    team = (dept == d)
    contacts = [m for n in team for m in n.getNeighbors()]
    showOnly(list(team) + contacts)
```

```
    summary.text = f"{d}: {len(team)} people, {len(interEdges(dept) & (team <->
↪   g.nodes))} ties outside"

def profile(n):
    reached = sorted(set(m.dept for m in n.getNeighbors()) - {n.dept})
    person.text = (f"{n.name} ({n.dept}): {n.degree} contacts in {len(reached)}
↪   other departments; "
                   f"effective size {n.effsize:.1f}, constraint
                    ↪  {n.constraint:.2f}")
    center(egoNetwork(n))

summary, person = Label(""), Label("click a person")
ui.dock(Panel("Brokers", Slider(0, 6, 2, label="other departments reached",
↪   onChange=brokers),
            Dropdown(sorted(dept.distinct), label="department", onChange=focus),
             Button("Everyone", onClick=lambda: (showAll(), brokers(2))),
↪   summary, person, position="bottom"))
graphevents.clickNode = profile
brokers(2)
center(giantComponent())
```

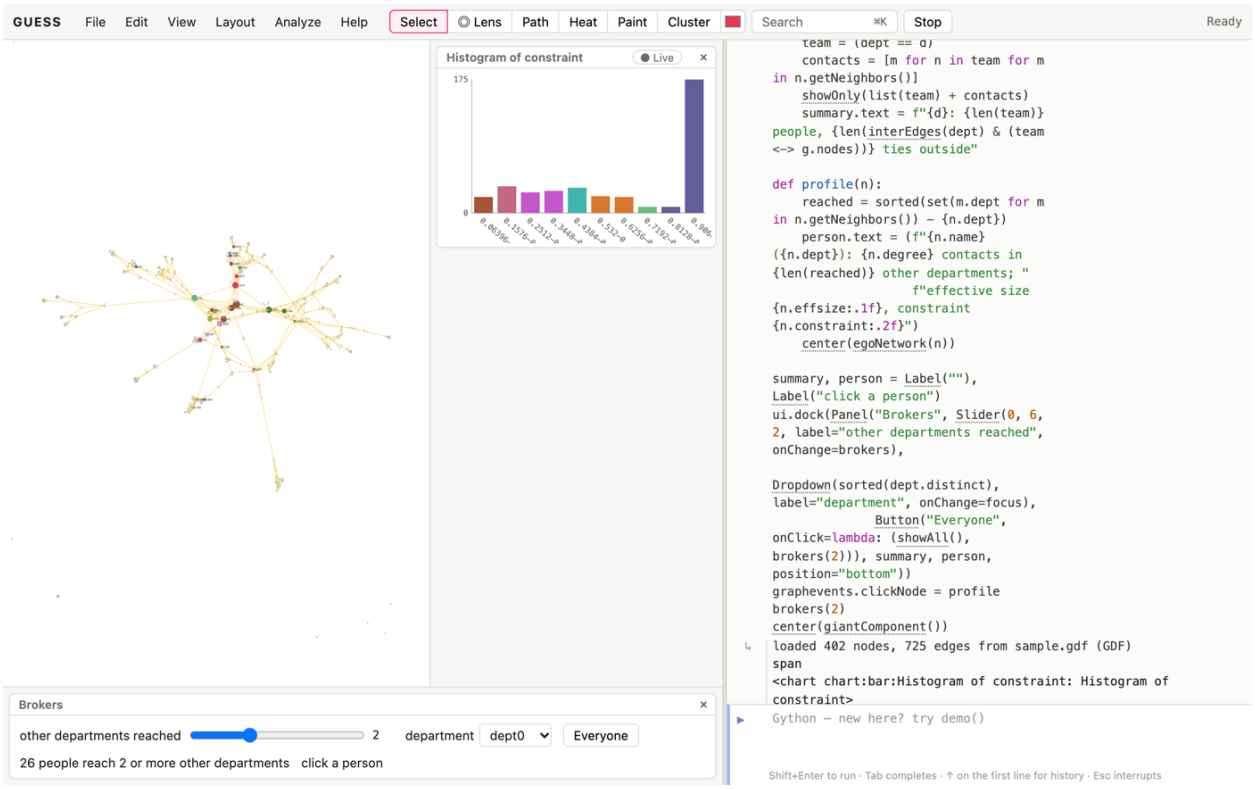


Figure 10: A broker finder: people sized by the number of other departments they reach

How it works.

- **The measure.** `span` counts, for each person, the departments their contacts belong to beyond their own: the raw material of brokerage (the brokerage recipe).
- **The slider.** It recolours and relabels so that only people above the threshold stand out. It doesn't hide anyone, so the brokers stay in context.
- **The drop-down.** It shows one department and everyone its members are tied to, with a count of the department's ties outside.
- **The click.** It reports the person's Burt measures, `effsize` and `constraint` (the ego-network recipe), and zooms to their ego network.
- **The chart.** `plotHistogram(constraint)` shows the distribution in a panel, so you can see where a given person falls.